

ISA 既有 RISC 指令集的属性, 也有 CISC 指令集的属性。然后, 将不同指令组织放到五个阶段中处理, 在此, 根据被执行的指令的不同, 每个阶段中的操作也不相同。据此, 我们构造了 SEQ 处理器, 其中每个时钟周期执行一条指令, 它会通过所有五个阶段。

流水线化通过让不同的阶段并行操作, 改进了系统的吞吐量性能。在任意一个给定的时刻, 多条指令被不同的阶段处理。在引入这种并行性的过程中, 我们必须非常小心, 以提供与程序的顺序执行相同的程序级行为。通过重新调整 SEQ 各个部分的顺序, 引入流水线, 我们得到 SEQ+, 接着添加流水线寄存器, 创建出 PIPE-流水线。然后, 添加了转发逻辑, 加速了将结果从一条指令发送到另一条指令, 从而提高了流水线的性能。有几种特殊情况需要额外的流水线控制逻辑来暂停或取消一些流水线阶段。

我们的设计中包括了一些基本的异常处理机制, 在此, 保证只有到异常指令之前的指令会影响程序员可见的状态。实现完整的异常处理远比此更具挑战性。在采用了更深流水线和更多并行性的系统中, 要想正确处理异常就更加复杂了。

在本章中, 我们学习了有关处理器设计的几个重要经验:

- 管理复杂性是首要问题。想要优化使用硬件资源, 在最小的成本下获得最大的性能。为了实现这个目的, 我们创建了一个非常简单而一致的框架, 来处理所有不同的指令类型。有了这个框架, 就能够在处理不同指令类型的逻辑中共享硬件单元。
- 我们不需要直接实现 ISA。ISA 的直接实现意味着一个顺序的设计。为了获得更高的性能, 我们想运用硬件能力以同时执行许多操作, 这就导致要使用流水线化的设计。通过仔细的设计和分析, 我们能够处理各种流水线冒险, 因此运行一个程序的整体效果, 同用 ISA 模型获得的效果完全一致。
- 硬件设计人员必须非常谨慎小心。一旦芯片被制造出来, 就几乎不可能改正任何错误了。一开始就使设计正确是非常重要的。这就意味着要仔细地分析各种指令类型和组合, 甚至于那些看上去没有意义的情况, 例如弹出值到栈指针。必须用系统的模拟测试程序彻底地测试设计。在开发 PIPE 的控制逻辑中, 我们的设计有个细微的错误, 只有通过控制组合的仔细而系统的分析才能发现。

网络旁注 ARCH:HCL Y86-64 处理器的 HCL 描述

本章已经介绍几个简单的逻辑设计, 以及 Y86-64 处理器 SEQ 和 PIPE 的控制逻辑的部分 HCL 代码。我们提供了 HCL 语言的文档和这两个处理器的控制逻辑的完整 HCL 描述。这些描述每个都只需要 5~7 页 HCL 代码, 完整地研究它们是很值得的。

Y86-64 模拟器

本章的实验资料包括 SEQ 和 PIPE 处理器的模拟器。每个模拟器都有两个版本:

- GUI(图形用户界面)版本在图形窗口中显示内存、程序代码以及处理器状态。它提供了一种方式简便地查看指令如何通过处理器。控制面板还允许你交互式地重新启动、单步或运行模拟器。
- 文本版本运行的是相同的模拟器, 但是它显示信息的唯一方式是打印到终端上。对调试来讲, 这个版本不是很有用, 但是它允许处理器的自动测试。

这些模拟器的控制逻辑是通过将逻辑块的 HCL 声明翻译成 C 代码产生的。然后, 编译这些代码并与模拟代码的其他部分进行链接。这样的结合使得你可以用这些模拟器测试原始设计的各种变种。提供测试脚本, 它们全面地测试各种指令以及各种冒险的可能性。

参考文献说明

对于那些有兴趣更多地学习逻辑设计的人来说, Katz 的逻辑设计教科书[58]是标准的人门教材, 它强调了硬件描述语言的使用。Hennessy 和 Patterson 的计算机体系结构教科书[46]覆盖了处理器设计的广泛内容, 包括这里讲述的简单流水线, 还有并行执行更多指令的更高级的处理器。Shriver 和 Smith [101]详细介绍了 AMD 制造的与 Intel 兼容的 IA32 处理器。

家庭作业

- 4.45 在 3.4.2 节中，x86-64 `pushq` 指令被描述成要减少栈指针，然后将寄存器存储在栈指针的位置。因此，如果我们有一条指令形如对于某个寄存器 `REG`，`pushq REG`，它等价于下面的代码序列：

```
subq $8,%rsp      Decrement stack pointer
movq REG, (%rsp)  Store REG on stack
```

- A. 借助于练习题 4.7 中所做的分析，这段代码序列正确地描述了指令 `pushq %rsp` 的行为吗？请解释。
- B. 你该如何改写这段代码序列，使得它能够像对 `REG` 是其他寄存器时一样，正确地描述 `REG` 是 `%rsp` 的情况？

- 4.46 在 3.4.2 节中，x86-64 `popq` 指令被描述为将来自栈顶的结果复制到目的寄存器，然后将栈指针减少。因此，如果我们有一条指令形如 `popq REG`，它等价于下面的代码序列：

```
movq (%rsp), REG  Read REG from stack
addq $8,%rsp      Increment stack pointer
```

- A. 借助于练习题 4.8 中所做的分析，这段代码序列正确地描述了指令 `popq %rsp` 的行为吗？请解释。
- B. 你该如何改写这段代码序列，使得它能够像对 `REG` 是其他寄存器时一样，正确地描述 `REG` 是 `%rsp` 的情况？

- 4.47 你的作业是写一个执行冒泡排序的 Y86-64 程序。下面这个 C 函数用数组引用实现冒泡排序，供你参考：

```
1  /* Bubble sort: Array version */
2  void bubble_a(long *data, long count) {
3      long i, last;
4      for (last = count-1; last > 0; last--) {
5          for (i = 0; i < last; i++)
6              if (data[i+1] < data[i]) {
7                  /* Swap adjacent elements */
8                  long t = data[i+1];
9                  data[i+1] = data[i];
10                 data[i] = t;
11             }
12     }
13 }
```

- A. 书写并测试一个 C 版本，它用指针引用数组元素，而不是用数组索引。
- B. 书写并测试一个由这个函数和测试代码组成的 Y86-64 程序。你会发现模仿编译你的 C 代码产生的 x86-64 代码来做实现会很有帮助。虽然指针比较通常是用无符号算术运算来实现的，但是在这个练习中，你可以使用有符号算术运算。

- 4.48 修改对家庭作业 4.47 所写的代码，实现冒泡排序函数的测试和交换(6~11 行)，要求不使用跳转，且最多使用 3 次条件传送。

- 4.49 修改对家庭作业 4.47 所写的代码，实现冒泡排序函数的测试和交换(6~11 行)，要求不使用跳转，且只使用 1 次条件传送。

- 4.50 在 3.6.8 节中，我们看到实现 `switch` 的一种常见方法是创建一组代码块，再用跳转表对这些块进行索引。考虑图 4-69 中给出的函数 `switchv` 的 C 代码，以及相应的测试代码。

用跳转表以 Y86-64 实现 `switchv`。虽然 Y86-64 指令集不包含间接跳转指令，但是，你可以通过把计算好的地址入栈，再执行 `ret` 指令来获得同样的效果。实现类似于 C 语言所示的测试代码，证明你的 `switchv` 实现可以处理触发 `default` 的情况以及两个显式处理的情况。